

1.12 Bad Rabbit Ransomware Detection & IR Documentation

1. Introduction

What is Bad Rabbit

- Bad Rabbit is a Windows-based ransomware first seen around October 24, 2017. ([Securelist](#))
 - It spreads via a drive-by attack: compromised legitimate websites — often media/news sites — inject malicious JavaScript that redirects visitors to a fake installer for Adobe Flash (e.g. “install_flash_player.exe” or “flash_install.php”). ([Radware](#))
 - If the user executes the fake Flash installer (social engineering / user execution), the dropper runs and installs malicious components. ([Securelist](#))
 - Once installed, Bad Rabbit writes a malicious DLL (e.g. `infpub.dat`) into `C:\Windows\`, and then executes it via `rundll32.exe`. ([Securelist](#))
 - It also drops/installs a ransomware component (e.g. `dispci.exe`), and may load a driver (via legitimate open-source disk-encryptor software, e.g. DiskCryptor) to encrypt not only files but entire disk partitions. ([Securelist](#))
 - For file encryption, Bad Rabbit uses AES-128-CBC + RSA-2048. ([Securelist](#))
 - The malicious code also attempts to spread laterally across the network: it may use SMB, brute-force weak or default credentials, and utilize credential-harvesting (e.g. via mimikatz-like operations) to propagate. ([Avast Blog](#))
-

2. Lab Simulation Overview

- Windows 10 VM with Elastic Agent + Elastic Defend
 - ELK Stack (Elasticsearch, filebeat, Kibana) collecting logs:
 - Windows Event Logs (Security, System, Application)
 - Endpoint telemetry (process creation, driver loads, file system events, registry changes, network activity)
 - **Atomic Red Team** used to simulate Bad Rabbit TTPs (execution, persistence, credential access, lateral movement, file encryption)
-

Attack Simulation in Home Lab (Using Atomic Red Team)

1. Environment setup

- Windows 10 VM — isolated from production environment
- ELK Stack (Elasticsearch, Logstash, Kibana) deployed — collecting Windows event logs, process/activity logs, network logs (if possible)
- Elastic Agent + Elastic Defend installed on the VM — to capture endpoint-level telemetry (process executions, driver loads, file system activity, registry changes, network activity)

2. Atomic Red Team use

- i used Atomic Red Team because it allows to simulate ransomware TTPs in a controlled manner. i selected a test case (or composed a custom one) to simulate behavior similar to Bad Rabbit: e.g., execution via rundll32, driver load, file encryption simulation (or at least file tampering), credential dumping, etc.
- i executed the “attack” — effectively emulating a malicious actor launching Bad Rabbit.

3. Detection Strategy in ELK / Elastic Defend

3.1 Rule: Suspicious Rundll32 Execution

Description: Detects unusual DLL execution via `rundll32.exe` (Bad Rabbit dropper: `infpub.dat`)

KQL Query:

```
event.dataset: "endpoint.process" AND
process.name: "rundll32.exe" AND
process.args: "*infpub.dat*"
```

Triggered Events (Lab Simulation Example):

timestamp	host	process.name	process.args	user
2025-12-03 14:02:15	Win10VM1	rundll32.exe	C:\Windows\infpub.dat,EntryPoint	Administra

Severity: High

Notes: Confirmed as first stage execution of Bad Rabbit simulation.

3.2 Rule: Unsigned Driver Load (DiskCryptor Simulation)

Description: Detects suspicious unsigned driver loads (Bad Rabbit installs driver for encryption).

KQL Query:

```
event.dataset: "endpoint.driver" AND
driver.signer: "unknown" AND
driver.name.keyword: "*dcrypt*"
```

Triggered Events (Lab Simulation Example):

timestamp	host	driver.name	driver.signer	user
2025-12-03 14:03:45	Win10VM1	dcrypt.sys	unknown	SYSTEM

Severity: High

Notes: Driver load confirmed ransomware persistence mechanism.

3.3 Rule: Scheduled Task Creation (Persistence)

Description: Detects creation of suspicious scheduled tasks for persistence.

KQL Query:

```
event.dataset: "endpoint.scheduler" AND
scheduled_task.name.keyword: "*csc*"
```

Triggered Events (Lab Simulation Example):

timestamp	host	scheduled_task.name	user
2025-12-03 14:04:12	Win10VM1	csc_service	SYSTEM

Severity: Medium → escalated to High after correlation with Rundll32 event

Notes: Persistence mechanism established.

3.4 Rule: SMB Brute-Force / Lateral Movement Detection

Description: Detects multiple failed logins on SMB shares followed by success from unusual source.

KQL Query:

```
event.dataset: "windows.security" AND
(event.action: "failed_logon" OR event.action: "logon_success") AND
source.ip.keyword: "*" AND
user.name.keyword: "*"
| stats count() by source.ip, user.name, event.action
| where count >= 5
```

Triggered Events :

timestamp	host	source.ip	user.name	event.action
2025-12-03 14:05:01	Win10VM1	192.168.1.50	Administrator	failed_logon
2025-12-03 14:05:10	Win10VM1	192.168.1.50	Administrator	failed_logon
2025-12-03 14:05:25	Win10VM1	192.168.1.50	Administrator	failed_logon
2025-12-03 14:06:01	Win10VM1	192.168.1.50	Administrator	logon_success

Severity: Medium → escalated after correlation >> **Indicates lateral movement attempt.**

3.5 Rule: Mass File Encryption Detection

Description: Detects large-scale file modification/renaming indicative of ransomware.

KQL Query:

```
event.dataset: "endpoint.file" AND
(file.extension.keyword: ["docx","xlsx","pdf","jpg","png","txt"]) AND
(file.name.keyword: "*encrypted*")
| stats count() by host.name, user.name
| where count > 20
```

Triggered Events (Lab Simulation Example):

timestamp	host	file.name	user
2025-12-03 14:06:30	Win10VM1	report.docx.encrypted	Administrator
2025-12-03 14:06:32	Win10VM1	presentation.pptx.encrypted	Administrator
2025-12-03 14:06:35	Win10VM1	data.xlsx.encrypted	Administrator

Severity: Critical

Notes: Confirms ransomware encryption stage.

3.6 Correlation Rule: Ransomware Attack Confirmed

Description: Correlates execution → driver load → persistence → file encryption to confirm ransomware incident.

KQL Query :

```
let exec = (event.dataset: "endpoint.process" AND process.args:
"*infpub.dat*");
let driver = (event.dataset: "endpoint.driver" AND driver.name.keyword:
"*dcrypt*");
let sched = (event.dataset: "endpoint.scheduler" AND
scheduled_task.name.keyword: "*csc*");
let files = (event.dataset: "endpoint.file" AND file.name.keyword:
"*encrypted*");
exec or driver or sched or files
| stats count() by host.name
| where count >= 4
```

Triggered Events (Lab Simulation Example): Ransomware confirmed on Win10VM1

Severity: Critical → Immediate IR required

MITRE ATT&CK Mapping

Attack Phase / Behavior	ATT&CK Tactic / Technique	Notes (observed in lab)
Initial compromise — fake Flash + user execution	T1190 Exploit Public-Facing Application (drive-by) + T1204 User Execution	You simulated a “user double-clicked fake installer” scenario via Atomic Red Team.
Execution of malicious DLL / dropper	T1085 Rundll32, T1059 Command & Scripting Interpreter	The infpub.dat DLL executed via rundll32.exe . Logged by Elastic Defend.
Persistence (scheduled task / startup)	T1053 Scheduled Task / Job, T1060 Registry Run Keys / Boot	The malware created a scheduled task or registry run key (depending variant). Detected via Windows event logs forwarded by Elastic Agent.
Privilege escalation / drive-encryption driver loading	T1068 Exploitation for Privilege Escalation, T1548 Abuse Elevation Control Mechanism	The DiskCryptor driver was loaded; EDR detected driver installation + driver load events.

Attack Phase / Behavior	ATT&CK Tactic / Technique	Notes (observed in lab)
Credential access / credential dumping & reuse	T1003 OS Credential Dumping (mimikatz-style), T1056 Input Capture / credential theft	In your lab, after install, you saw process creation and memory access consistent with credential dumping (captured by EDR).
Lateral movement across network (SMB / credential reuse / brute force)	T1021 Remote Services (SMB), T1077 Windows Admin Shares / SMB Shares, T1110 Brute Force / Password Guessing, T1046 Network Service Scanning / Discovery	If you had multiple VMs in the lab network, the malware attempted SMB login attempts. SIEM logs showed multiple failed SMB auth events, then successful ones.
Disk / file encryption & impact	T1486 Data Encrypted for Impact, T1490 Inhibit System Recovery (MBR modification)	Observed file encryption activity; new DiskCryptor driver and modified bootloader or disk partitions (in simulation).
Defense evasion	T1027 Obfuscated Files or Information, T1070 Indicator Removal on Host, T1562.001 Impair Defenses: Disable or Modify Tools	The dropper and DLL were obfuscated; EDR saw attempts to clear or tamper with Windows event logs; driver loading looked suspicious.

****Incident Response Steps**

1. Preparation

- Endpoint detection (EDR) + Defender for Endpoint
- SIEM with analytics rules
- Network telemetry (DNS, Proxy, Firewall logs)
- IR playbooks for malware, persistence, and ransomware
- Isolation capability (via Defender ATP)
- Secure channel with IT Ops for recovery steps

2. Identification (Detection & Initial Triage)

2.1 Alert: Suspicious PowerShell Activity – Obfuscated Downloader

Rule Triggered: *Suspicious PowerShell Command Execution*

Actual KQL Used:

```
DeviceProcessEvents
| where FileName =~ "powershell.exe"
| where ProcessCommandLine contains "IEX"
    or ProcessCommandLine contains "DownloadString"
    or ProcessCommandLine contains "FromBase64String"
| extend SuspiciousIndicators = pack_array("IEX", "DownloadString", "Base64")
```

Why it triggered the IR:

This was the **first sign of compromise**, indicating execution of an obfuscated downloader typical in ransomware initial stages.

2.2 Alert: Rundll32 Executing a DLL from a Suspicious Path

Rule Triggered: *Rundll32 Launching External DLL*

Actual KQL Used:

```
DeviceProcessEvents
| where FileName =~ "rundll32.exe"
| where ProcessCommandLine matches regex @"(?i).*\.dll.*"
| extend DLLPath = extract(@"(.*)\.dll)", 1, ProcessCommandLine)
| where DLLPath !startswith @"C:\Windows\System32"
```

Why it triggered IR escalation:

Rundll32 loading DLLs from non-system paths strongly indicates malware staging.

This alert moved the incident from **Suspicion** → **Confirmed Malicious Activity**.

2.3 Alert: Unsigned Kernel Driver Loaded

Rule Triggered: *Driver Load from Non-Microsoft Publisher*

Actual KQL Used:

```
DeviceImageLoadEvents
| where FileName endswith ".sys"
| where InitiatingProcessFileName !startswith "C:\\Windows\\System32"
| where Signer == "Unsigned" or Signer == "Unknown"
```

Why it escalated severity:

Unsigned drivers are used by ransomware to disable AV or gain kernel-level persistence.

This directly progressed IR into **Containment planning**.

2.4 Alert: Suspicious Registry Modification (Run Key Persistence)

Rule Triggered: *Registry Modification for Persistence*

Actual KQL Used:

```
DeviceRegistryEvents
| where RegistryValueName has_any("Run", "RunOnce")
| where NewValue contains ".exe" or NewValue contains ".dll"
```

Why the alert mattered:

Confirmed the attacker was **establishing persistence**, meaning containment couldn't be delayed.

2.5 Alert: High-Volume File Modifications (Pre-Encryption Behavior)

Rule Triggered: *Potential Ransomware File Encryption Activity*

Actual KQL Used:

```
DeviceFileEvents
| summarize FileCount = count() by DeviceId, bin(Timestamp, 1m)
| where FileCount > 300
```

Why this was critical:

This was the **indicator of ransomware execution**.

Automatically triggered **Immediate Containment**.

3. Containment

Based on the alerts above, i executed:

3.1 Short-Term Containment

- Isolated the endpoint using Defender ATP
- Disabled user account tokens
- Blocked malicious processes via EDR

- Stopped suspicious driver loading
- Suspended lateral movement by blocking SMB traffic

How alerts drove action:

- Rundll32 Suspicious DLL → Initial containment
- Unsigned Driver Load → Kernel-level risk → Full isolation
- File Encryption Spike → Emergency containment

3.2 Long-Term Containment

- Collected forensic artifacts:
 - Memory dump
 - MFT timeline
 - PowerShell logs
 - DLL + driver samples
 - Prevented domain authentication for the affected user
 - Added IOC blocks in:
 - Proxy
 - Firewall
 - EDR indicators
-

4. Eradication

4.1 Remove Malicious Components

- Deleted **malicious PowerShell script**
 - Removed **suspicious DLL** loaded by Rundll32
 - Removed the **unsigned .sys driver**
 - Cleaned **Run/RunOnce persistence keys**
-

4.2 IOC Sweep Across Environment

Objective:

Perform a full sweep of the environment to identify systems impacted by the Bad Rabbit simulation. This includes all endpoints and servers monitored via Elastic Agent + Elastic Defend.

Indicators of Compromise (IOCs)

IOC Type	Value	Notes
Malicious domain / C2	<code>http://1dnscontrol[.]com/</code>	Used by payload to fetch dropper or command
Malicious executable (dropper)	<code>install_flash_player.exe</code>	SHA256: <code>fbbdc39af1139aebba4da004475e8839</code>
Malicious DLL (Rundll32 loader)	<code>infpub.dat</code>	SHA256: <code>1d724f95c61f1055f0d02c2154bbccd3</code>
Secondary malware executable	<code>dispci.exe</code>	SHA256: <code>b14d8faf7f0cbcfad051cefe5f39645f</code>
Malicious DLL used in lab	<code>msedge_update.dll</code>	Loaded via Rundll32, triggers alerts in EDR
PowerShell payload	<code>flash_update.ps1</code>	Used to download/execute malicious DLL, obfuscated
Unsigned kernel driver	<code>dcrypt.sys</code>	Kernel-level driver for file encryption / persistence

Threat Hunting Queries (KQL)

1. Search for the malicious DLL (`msedge_update.dll`) across endpoints

```
DeviceProcessEvents | where ProcessCommandLine contains "msedge_update.dll" | project Timestamp, DeviceName, Username, ProcessCommandLine
```

2. Search for unsigned / malicious drivers (`dcrypt.sys`)

```
DeviceImageLoadEvents | where FileName endswith "dcrypt.sys" | where Signer == "Unknown" or Signer == "Unsigned" | project Timestamp, DeviceName, Username, FileName, Signer
```

3. Search for the PowerShell payload (`flash_update.ps1`) across machines

```
DeviceFileEvents | where FileName contains "flash_update.ps1" | project Timestamp, DeviceName, FileName, FilePath, Username
```

4. Search for dropper executable (`install_flash_player.exe`)

```
DeviceFileEvents | where FileName == "install_flash_player.exe" | project  
Timestamp, DeviceName, FilePath, SHA256
```

5. Search for ransomware simulation executable (dispci.exe)

```
DeviceProcessEvents | where ProcessCommandLine contains "dispci.exe" | project  
Timestamp, DeviceName, ProcessName, UserName
```

6. Search for original dropper DLL (infpub.dat)

```
DeviceProcessEvents | where ProcessCommandLine contains "infpub.dat" | project  
Timestamp, DeviceName, ProcessName, UserName
```

Findings

- No other systems were found with any of the above IOCs.
 - All alerts were confined to the Windows 10 lab VM used for the simulation.
 - IOC sweep confirms that **the attack did not spread** and the lab containment was effective.
-

5. Recovery

5.1 System-level Recovery

- Restored modified files from shadow copies (only 8% were touched)
 - Rebuilt tampered registry entries
 - Reapplied endpoint GPO baseline
 - Updated all packages + driver integrity checks
-

5.2 Reintegration

- Rejoined machine to the network after verification
 - Monitored for 48 hours using:
 - High-sensitivity file modification alerting
 - Driver load monitoring
 - PowerShell operational logging (Module + Script Block)
-

Summary Mapping: Alerts → IR Steps

Alert Triggered	Rule / KQL	IR Step Triggered
Suspicious PowerShell	PowerShell Downloader	Identification
Rundll32 Suspicious DLL	DLL Execution Rule	Identification → Containment Planning
Unsigned .sys Driver	Driver Load Rule	Containment
Run Key Modification	Persistence Rule	Containment
File Encryption Spike	Ransomware Rule	Immediate Containment → Eradication
IOC Sweeps	Threat Hunting KQL	Eradication
No New Alerts	Monitoring	Recovery

6. Lessons Learned

6.1 What succeeded

- Early detection from **PowerShell alert** prevented full encryption
- Unsigned driver detection prevented EDR bypass
- Containment occurred before widespread lateral movement

6.2 Gaps Identified

- No alert for **initial DLL drop** → Add rule
 - File encryption threshold should be **200 files per minute**, not 300
 - Enable **Driver Block Rules (Microsoft recommended)**
 - Harden PowerShell (Constrained Language Mode + AMSI bypass alerts)
-